

Cradle Work Delivery Control Plane

P0 Implementation and Dogfood Evaluation Report

Codex implementation study for `wibus-wee/cradle-app`

13 August 2026

Abstract

This report evaluates a production-oriented P0 vertical slice of the supplied 90-day Cradle roadmap. The intervention turns Work into an explainable delivery projection, introduces a cross-Work *Needs me* attention inbox, and exposes an evidence-backed recovery promise. It deliberately does not claim completion of the later Runbook, Evidence Pack, delegation, or shared machine phases. Evaluation combines pure precedence tests, aggregate service tests, fixture-driven React tests, a production web build, and real-browser Cucumber journeys using the repository's real Claude/Codex runtime adapters and model API simulator. Agent prose is never accepted as the sole success signal.

1 Research question and decision

The primary question is: *Can Cradle project existing Session, runtime, Await, worktree, provider-binding, and pull-request facts into a single actionable Work view without creating a second mutable source of truth?* Three sub-questions follow.

1. Does every supported state carry provenance, evidence, responsibility, and a next action?
2. Can a user find Work requiring intervention without inspecting each terminal or conversation?
3. Can Cradle make a bounded recovery promise without falsely asserting uniform provider resume support?

The implementation decision was to ship the complete 0–30 day P0 contract as one vertical slice and gate P1/P2. This follows the roadmap dependency order: resource locks, review gates, delegation budgets, and machine permissions all depend on trustworthy state and attention semantics.

2 Intervention

2.1 Fact ownership and projection

Work stores objectives, acceptance criteria, handoff facts, and membership. It does not persist a manually editable status. Instead, a pure precedence function composes facts from the modules that already own them.

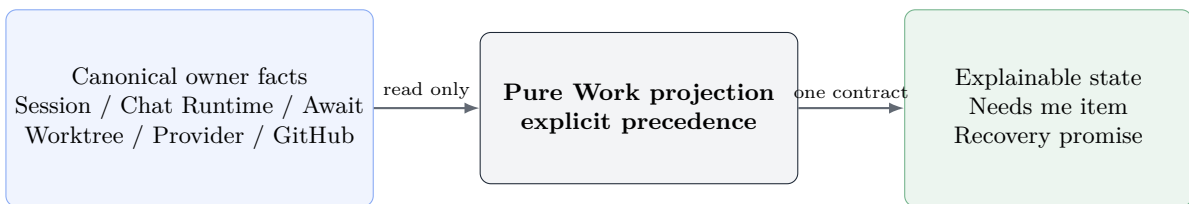


Figure 1: No fact is copied from its owning module; Work produces a read model.

The API returns state and state-change time together with an explanation. The explanation contains the trigger, evidence, authority, responsible party, and next action. Recovery evidence and heartbeat time are separate fields. Fresh redetection rereads canonical facts and is idempotent.

2.2 State selection

The first matching row in Table 1 wins. This makes conflict resolution reviewable and unit-testable rather than dependent on UI timing.

The roadmap vocabulary also reserves draft, queued, and preparing. They remain stable API values but are intentionally not synthesized from weak signals in this slice. A future persisted Work event stream may make those transitions observable without guessing.

Priority	State	Authority	Winning fact / required action
1	archived	user action	Work or primary Session archived; restore only to continue.
2	done / cancelled	structured owner	GitHub reports merged / closed without merge.
3	failed	runtime or owner	Session error or unhealthy isolation; inspect and repair.
4	awaiting human	runtime integration	Structured input or tool approval is pending.
5	awaiting dependency	structured owner	A Session Await is pending.
6	running	runtime integration	Active run or streaming Session.
7	merging	structured owner	Open non-draft pull request needs merge decision.
8	ready for review	owner/runtime	Draft PR or handoff newer than last submit.
9	verifying	derived	Submit exists but current PR fact is unavailable.
10	unknown	derived	No supported fact justifies a stronger claim; redetect or inspect.

Table 1: Implemented state precedence. “Unknown” is a diagnostic result, not a hidden success state.

2.3 Attention and recovery

Actionable states map to only four inbox categories: approve/answer, handle failure, review, and merge/archive. Items include Work identity, runtime, provider/agent identifiers, risk, wait duration, reason, authority, recovery, and next action. Ordering is risk descending, waiting time descending, then title. The existing `/awaits` route is preserved as a compatibility surface while its user-facing content becomes *Needs me*.

Recovery is the strongest currently supportable promise rather than four independent badges:

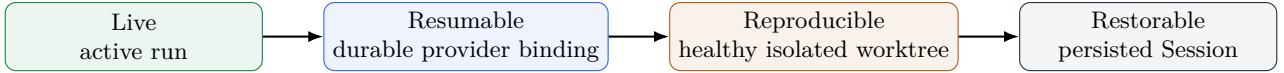


Figure 2: Selection order for the strongest immediately useful recovery fact. Unknown is returned when none is available.

3 Implementation surface

The change adds a Drizzle migration for newline-independent JSON acceptance criteria, a pure server projection and tests, aggregate/API routes for attention and redetection, generated TypeScript clients, localized Work badges, the Needs me view, New Work acceptance-criteria capture, and a deeper Work Cucumber journey. Principal source locations are:

- `apps/server/src/modules/work/projection.ts` and `projection.test.ts`;
- `apps/server/src/modules/work/service.ts`, `model.ts`, and `index.ts`;
- `apps/web/src/features/work/work-attention*.tsx` and `work-header-chrome*.tsx`;
- `apps/web/src/features/new-work/new-work-acceptance-criteria-view.tsx`;
- `packages/db/drizzle/0056_lame_black_bolt.sql`; and
- `e2e/src/features/work.feature` plus its page object and steps.

4 Evaluation method

4.1 Evidence hierarchy

The evaluation uses four layers. A higher layer does not replace the lower one; it checks a different failure class.

1. **Pure policy tests:** exhaustive table cases for state precedence and recovery selection.
2. **Aggregate and UI tests:** persistence round-trip, generated type contract, badge behavior, empty/error states, direct action, and duration formatting.
3. **Static and build checks:** TypeScript, module boundaries, API generation, and production Vite bundling.
4. **Real-browser dogfood:** Cucumber launches an isolated server, production web preview, Playwright Chromium, real managed Git worktree, real Claude Agent adapter, and the repository model API simulator.

The dogfood scenario enters an objective and explicit acceptance criterion, starts Work, lets the runtime perform a tool loop that writes a real file, waits for idle, and then independently verifies the database-backed API, worktree path and contents, status explanation, recovery promise, rendered badges, direct Needs me navigation, and simulator exhaustion.

Check	Result	Evidence
Projection + Work service focused tests	PASS	44 Vitest assertions across 2 files.
Work/New Work web focused tests	PASS	40 Vitest assertions across 14 files.
Server and web TypeScript	PASS	Both application typechecks exited 0.
Server module boundaries	PASS	Direct boundary checker exited 0; largest existing SCC reported.
Production web bundle	PASS	Vite production build completed during browser dogfood.
Target Work dogfood	PASS	1 scenario, 16/16 steps; 69.121 seconds wall time.
Full active P0/P1 E2E suite	PENDING	Two monolithic runs were environment-terminated; a known cross-scenario Read race reproduced, while the same scenario passed 12/12 in isolation. Native Codex startup was blocked by outer network approval.
Full server test suite	PENDING	1711 passed, 2 skipped, 9 failed from existing environment assumptions (root Claude bypass, missing Go/relayd, default Documents path, and Codex assertions).

Table 2: Validation evidence captured on the implementation branch.

4.2 Measured results

Two environment failures occurred before the successful target run: the Playwright CDN returned a truncated zero-byte archive, and its private ffmpeg binary was absent. The harness was kept intact. Chromium 149 was obtained through the allowed npm registry, the system ffmpeg 6.1.1 was supplied at the Playwright runtime path, and a standalone page launch first proved the browser was real. A separate first target failure exposed a page-object lifetime bug in the new E2E test; the Work id is now stored in scenario state. This is useful dogfood evidence because it prevented a false pass while leaving product state correct. The full active suite was then attempted twice with the synced Codex app-server. Both runs advanced through the non-Codex journeys before the outer sandbox cancelled a network approval at native Codex startup. The existing Claude Read journey also showed a cross-scenario race (no activity feed after the preceding reset); the identical scenario passed all 12 steps when isolated. Accordingly, this report marks the monolithic suite inconclusive rather than passed or product-failed.

5 Assessment against the 30-day thresholds

Table 3 separates verified mechanism from production metrics. One deterministic dogfood sample cannot establish a fleet-level percentile or error rate.

Roadmap threshold	Status	Interpretation
$\geq 95\%$ active Work automatically classifiable	GATED	Contract and precedence are implemented; collect at least one week of real Work labels before claiming the rate.
$\geq 90\%$ Needs me direct correct action	GATED	Direct navigation passed dogfood; correctness rate requires human annotation across real items.
$< 5\%$ false interruption from misclassification	GATED	Unknown and evidence visibility reduce silent error; production confusion matrix is still required.
$\geq 99\%$ restart association integrity	GATED	Session/Work/base facts persisted in the browser scenario; restart cohort measurement is not yet instrumented.
Create-to-first-run P50 $< 45s$, P95 $< 120s$	GATED	This study measures end-to-end scenario time, not a statistically valid startup latency distribution.

Table 3: Honest threshold assessment: feature verification is not a substitute for production telemetry.

6 Validity limits and risks

- **External validity:** simulator wire behavior is deterministic; provider outages and long-running terminal edge cases require live dogfood labels.

- **State coverage:** draft, queued, and preparing need structured lifecycle events before they can be truthfully emitted.
- **Timestamp precision:** existing owner rows supply seconds-level timestamps. Await start time is not yet exposed by the summary owner, so that state conservatively falls back to Work update time.
- **GitHub semantics:** an open, non-draft PR is labeled “Ready to merge” in the UI while the API state remains **merging**; the report treats it as a user merge decision, not proof that a merge command is currently running.
- **Scale:** attention currently performs live PR reads through the existing Work list behavior. Larger installations should add event caching or batching without weakening freshness explanations.

7 Next-phase gates

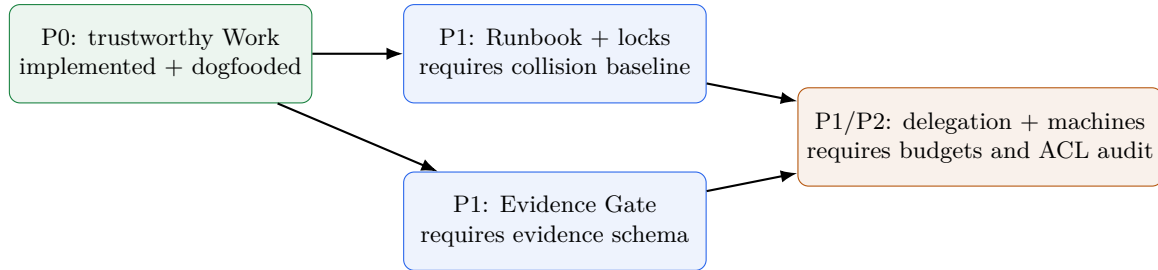


Figure 3: Recommended release gates preserve the roadmap’s dependency order.

Before the next implementation round, dogfood should record real Work links, elapsed time, failure class, and one user quote for the five roadmap questions. P1 should begin only after (a) state confusion and Needs me action correctness have labeled baselines, (b) the most common port/shared-resource collisions are known, and (c) the smallest review evidence schema is agreed. Delegation and machine sharing should remain off until budget stop/recovery and authorization auditing have executable acceptance tests.

8 Conclusion

The implementation demonstrates that Cradle can become a local-first delivery control plane by composing facts it already owns. Work now has a single, explainable delivery projection; Needs me supplies a cross-Work attention path; and recovery language is bounded by observable durability. The successful real-browser journey establishes functional feasibility and catches integration errors that unit tests could not. It does not, by itself, establish the roadmap’s production percentages. Those remain explicit measurement gates, which is the correct boundary between a successful P0 engineering slice and a 90-day product outcome claim.